Supplementary Material for "RAI: Safe Payments Through Committee Changes Without Stopping"

Anonymous Author(s)

Regular paper (supplement, not part of the page budget)

This supplement complements the submission. Appendices A to C give the formal statement of the protocol that Sections III to VII of the main text describe in words: the model and the two abstract services with their properties C1–C2 and A1–A4, the account-local rules, and the epoch transition with the report format, eligibility, the construction BuildState, and installation. Appendix D gives the full proofs of the results of Section VIII of the main text and the counterexample behind the early-vote rule. Appendix E describes the six TLA$^+$ models, their abstractions, configurations, and results. Appendix F lists the scripted regression cases derived from the models and their status in the prototype. Appendix G records the prototype's departures from the protocol, the defects found while measuring, the additional experiments (late notarization, committee change, restart and durable signing, handoff), and how to reproduce the runs. Theorem and lemma numbers are those of the main text; Theorem 3 of the main text is Theorems 3 and 4 here. Throughout, $N = 3f + 2p + 1$, $q = N - f - p$, $r = f + p + 1$, and reports are selected in sets of $N - f$.

Appendix A
Formal Model, Assumptions, and Abstract Services

This appendix and the two that follow are the formal counterpart of Sections III to VII of the main text: the same rules, stated with the notation the proofs use.

A. Participants, faults, and retained evidence

Clients own accounts and sign blocks. An authenticated committee contains $N = 3f + 2p + 1$ equal-weight validator identities, $f, p \geq 0$, of which at most f are Byzantine. Corruption is static: adversarial identities are fixed throughout the execution, and a correct departing identity is not later compromised. Membership admission, Sybil resistance, and the per-committee fault bound are deployment assumptions.

Signatures are unforgeable and hashes collision resistant; canonical encodings bind the network session and message domain. An account vote binds its epoch, committee, account, slot, vote type, block hash, and admission-witness digest, where the witness identifies the anchor and evidence under which the vote was admitted, including its base checkpoint (Section B-B); checkpoint votes bind the handoff context and deciding committee. Distinct signatures by one identity never contribute twice to a quorum for the same block.

TABLE I
Account and closure thresholds. The example has $f = p = 1$, $N = 6$.

Evidence	Threshold	Example
Notarization NC / exclusion witness XW	$q = N - f - p$	4
Normal finality FC	q final votes	4
Fast finality FF	$N - p$ settled first votes	5
Recovery support	$r = f + p + 1$	3
Selected reports	$N - f$	5

Correct validators persist their first vote and terminal state before releasing signatures. They retain blocks, signed votes, and finite dependency evidence, and relay them after local voting ends; a certificate is a deterministic local view of sufficient individually verified votes, not a separately gossiped object. A retained item held by a correct validator eventually reaches every correct validator that needs it (the *data service*), and departing validators continue service until correct successors hold durable copies.

B. Quorums, accounts, and canonical state

The account path uses $N = 3f + 2p + 1$, $q = 2f + p + 1$, and $r = f + p + 1$ (Table I). The parameter p is the number of first-vote responses the fast path can omit, not an additional fault allowance. We repeatedly use

$$2q - N = f + 1, \quad q - 2f = p + 1, \quad (N - p) - 2f = r. \tag{1}$$

Each account is a tree of owner-signed blocks rooted at genesis. A block's slot is one greater than its parent's slot; slots are account positions, not times. Blocks of one account conflict when neither is an ancestor of the other. An owner equivocates by signing conflicting blocks; safety covers such owners, liveness does not promise a winner under persistent contention.

A closed checkpoint is $S_e = (L_e, \Sigma_e)$. The ledger L_e identifies finalized prefixes, retained unresolved branches, and lock records. A lock record names its block, its strength (notarization or recovery), and its *origin*: the epoch whose potential finality it protects. A block may carry several recovery records, at most one per origin. Each record is discharged separately,

and only by evidence of its own origin epoch; the block stays retained while any of its records survives. The application state Σ_e contains balances, received-send identifiers, and applied-operation identifiers. A block is final either from an eligible account FC/FF or because a decided checkpoint promoted it under Section C-E; an undecided candidate has no finality effect.

Let F_e be the finalized block set of L_e. A validator also maintains a certified live overlay O_i of locally finalized blocks not yet in F_e; its effective final set is $F_e \cup O_i$, and its live view is the deterministic replay of that set. Section C-F defines installation without rolling back or reissuing effects. A deterministic function derives the committee C_e from Σ_e, never from a private overlay.

C. Latent finality and progress assumptions

For a fixed epoch and admission context, a block is *latent-final* at a cut if released signatures and valid dependency evidence suffice to verify an eligible FC or FF for it or for a descendant selecting it; *late* finality is such a proof assembled or exposed after reports freeze or after a checkpoint is decided. A closure obligation also covers a certificate completed by subsequent old-epoch signatures from identities that have not stopped. Safety has no message-delay bound. Progress requires finite relevant admitted work, fair processing and retry, retained-data availability, and recurring sufficiently long bounded-delay windows in which δ bounds delivery and processing and clock rates lie in $[1-\rho, 1+\rho]$. An uncontested continuation has no conflicting owner-signed alternative at any new position, including signatures created earlier.

D. The two abstract services

RAI commits to sets by their roots and agrees on candidates by their digests. Two services turn these commitments into shared state; RAI states their interfaces and the properties it uses, not their mechanics.

Convergence. A reporter calls $\mathsf{Freeze}_i(\mathsf{ctx}_e, T_i, G_i)$, which returns the signed header R_i of Section C-B and retains the frozen sets. Any validator calls $\mathsf{Reconstruct}_j(R_i)$, which returns (T, G) or is still pending.

C1 (Integrity). If $\mathsf{Reconstruct}_j(R_i)$ returns (T, G) at a correct j, then $\mathsf{Root}(T)$ and $\mathsf{Root}(G)$ are the roots signed in R_i. With collision resistance, $(T, G) = (T_i, G_i)$ when i is correct, and a Byzantine i is bound to one pair per header.

C2 (Termination). If a correct i froze R_i and R_i reaches a correct j, then under the data service and fair retry $\mathsf{Reconstruct}_j(R_i)$ eventually returns.

Full transfer of the committed sets, an authenticated difference against a shared projection, and set reconciliation all satisfy C1 and C2; they differ in cost. A Byzantine reporter can make only its own report unreconstructible.

Consensus. For handoff e, members of the closing committee K_e call $\mathsf{Propose}(\mathsf{ctx}_e, v)$ with candidates $v = (Q, \mu, d)$ of Section C-D, and $\mathsf{Decide}(\mathsf{ctx}_e)$ returns a value v with a proof π_e that $\mathsf{Verify}(\mathsf{ctx}_e, v, \pi_e)$ accepts at any party knowing K_e. RAI supplies the predicate $\mathsf{Eligible}_e(v)$ of Section C-D, which a correct member can evaluate once it has reconstructed the reports v names.

A1 (Agreement). Two accepted proofs for ctx_e carry the same v.

A2 (Eligibility). An accepted proof for ctx_e carries a v with $\mathsf{Eligible}_e(v)$: a correct member supports a value only after checking $\mathsf{Eligible}_e$ on reconstructed inputs, and no decision forms without correct support.

A3 (Integrity). A proof cannot be replayed for another context, and Verify needs only K_e and the retained inputs v names.

A4 (Termination). If at least $N - f$ correct members of K_e eventually hold some eligible candidate for ctx_e, then every correct member eventually decides, given recurring prepared windows long enough for the service's own rounds.

A2 is external validity [**?**] with a predicate the service treats as opaque. C2 feeds A4: convergence makes correct reports usable, usable reports make an eligible candidate constructible at every correct member, and the consensus service then decides it. Nothing in A1–A4 refers to accounts, blocks, or report contents; any Byzantine agreement protocol with external validity and verifiable decisions, leader-based or not, fits the interface. Safety (Appendix D) uses C1 and A1–A3 together with the static fault bound, authentication, persistent voting, and the voting rules of Section B; termination additionally uses C2, A4, the data service, finite preparation, and recurring prepared windows.

Appendix B
Formal Account-Local Rules

A. Blocks, operations, and validation

A block is $B = (a, h_p, x, \sigma_a)$: account, parent hash, ordered operation batch, and owner signature, with identifier $h(B) = H(\mathsf{Block}, a, h_p, x)$. The epoch is absent from the identifier, so that a nonfinal omitted block can be retried with new-epoch votes; an empty batch is permitted for owner-authorized branch continuation. The operations are $\mathsf{Send}(b, z)$, for destination b and positive amount z, and $\mathsf{Receive}(s)$, where $s = (h(B), j)$ identifies a send at operation index j. A send debits its owner; a receive credits only the named destination and requires an explicit proof that the send is already final.

Validation replays the entire unresolved selected path from its finalized anchor, then the proposed batch, checking signatures, consecutive parent slots, amounts, balances, and duplicate receives; it has no application effect, and every finalization repeats the checks and applies only unapplied operations. A finality proof is an account-certificate proof with the evidence needed to verify eligibility, or a verified decided checkpoint that promoted the block. A proof used for epoch-e voting cannot depend on S_e or a later checkpoint, and a receive's send proof must be available before a correct validator supports the receiving path, which makes replay dependencies well founded (Lemma 7).

B. Single-support voting and durable state

An account instance is $(session, e, a, v)$. A correct validator first-votes at most once in that instance, for the first admissible available block it accepts; there is no second-look support for a rival and no account timeout vote.

Definition 1 (Support, notarization certificate, exclusion witness): A member of K_e *supports* a block B in an instance if it signed an epoch-e first vote for B or an epoch-e late notarization for B; a correct validator supports at most one block per instance. A *notarization certificate* $\mathsf{NC}_e(B)$ is a set of q epoch-e *first votes* for B from distinct members. An *exclusion witness* $\mathsf{XW}_e(B)$ is a set of q epoch-e supports for B from distinct members, first votes or late notarizations in any mix. Every $\mathsf{NC}_e(B)$ is an $\mathsf{XW}_e(B)$; the converse fails.

An NC of first votes is what makes a block complete, what a validator records before final-voting, what an N report entry claims, and what Rules 1 and 3 of Section C-E consume; an XW serves only as the closing-epoch evidence of the overlap exception, as the discharge of a recovery record of origin e on a conflicting block, and inside the overlap certificate behind an F claim. Letting late notarizations count toward an NC makes correct reports unusable (Appendix E).

A block is *complete* in epoch e when its data and required ancestry are available and the validator has assembled its NC_e. A complete eligible block is durably recorded as notarized before the validator final-votes, uses it as a complete parent, or freezes its report. The validator can final-vote only for its own first-voted block, and atomically records a terminal flag with that signature.

Certificates and the early-vote rule. Every first vote names its *base*, the last checkpoint its signer had installed when signing; the admission-witness digest binds it. An epoch-e first vote is *settled* if its base is S_{e-1} and *early* otherwise. An eligible block finalizes on an $\mathsf{FC}_e(B)$ of q final votes or an $\mathsf{FF}_e(B)$ of $N - p$ *settled* first votes; early first votes count toward $\mathsf{NC}_e(B)$ and $\mathsf{XW}_e(B)$ and toward nothing else. A validator that has not installed S_{e-1} releases an epoch-e final vote for B only while it holds, for B and every unresolved block of its selected prefix, the closing-epoch exclusion witness and the current-epoch NC that the overlap exception requires, and it retains them with its notarization record; after installing S_{e-1} it final-votes under ordinary rechecking. A block first-voted during a close therefore finalizes in two rounds rather than one. Without the rule, a fast certificate can be formed from early votes whose eligibility rests on a witness their signers never held, and the next closure cannot see it (Appendix E).

A finalized block's selected unresolved ancestors finalize with it, and incompatible provisional branches leave the effective live view; an NC alone has no balance effect.

Cross-epoch voting record. A correct identity stops epoch-$(e-1)$ first and final voting before opening epoch e. While S_{e-1} is unknown, it does not first-vote for a different block at a position where it supported a block in epoch $e-1$; re-voting the same block is allowed. Once the predecessor is installed, its finalized history, frontier, and carried lock records govern the position.

Termination rules. Two further rules make every forked position final (Theorem 4); both only withhold votes. *Checkpoints decide:* once a validator has installed a checkpoint that holds an unresolved block K at (a, v), it casts no new first, late, or final vote at (a, v) for a block whose branch bypasses a surviving record of that checkpoint, unless the bypassing block at the divergent position is final at the validator. An exclusion witness that discharges the record still makes the rival admissible as an instance, lets the validator accept a rival's overlap certificate (Lemma 8), and discharges the record inside a manifest; it no longer earns the rival a vote. *No early votes at a contested position:* a position is *contested* at a validator once it has seen two conflicting blocks there or its latest installed checkpoint holds an unresolved block there; at a contested position a validator casts no new epoch-e first vote before it has installed S_{e-1}. Re-broadcasting a vote already cast is allowed.

Late notarization. A member of K_{e-1} keeps notarizing for its closing epoch until it installs S_{e-1}. When it first-votes B at (a, v) in epoch e while S_{e-1} is unknown, it also signs a notarization-only epoch-$(e-1)$ vote for B, unless it supported a different block at (a, v) in epoch $e - 1$; if it first-voted B itself there, that vote already supports B. The late vote is durably recorded before release. It counts toward an epoch-$(e - 1)$ exclusion witness and toward nothing else: no NC, FF, or FC counts it, no final vote and no N entry follows from it, and no report contains it, since reports are frozen before it is cast. Late notarizations are retained evidence that a manifest may carry, but only inside an exclusion witness. They give a block published after the boundary the closing-epoch exclusion witness that the overlap exception requires.

C. Attachment and prefix-wide eligibility

Let $d_a(S)$ be the maximum depth of any block for account a retained in checkpoint S. With S_{e-1} installed, a retained tip continues at its own depth (Section C-F), and a *new* epoch-e root starts at depth $d_a(S_{e-1})+1$ on a maximum-depth retained tip. Every parent, a retained tip included, must be *settled for extension* in epoch e: final, held under a notarization lock (as its target or as an ancestor of the target), or complete in epoch e. A tip held by recovery records alone is therefore a parent in epoch e only once the validator holds an epoch-e NC of first votes for it, assembled in its own instance at its own position, which it already has (Section C-F). Final blocks and blocks under a notarization lock need nothing new, because no rival is ever admitted past them. An extension must respect finalized history and every carried lock record on its selected ancestry. Before S_{e-1} is known, first voting may prepare work from a parent final in the latest closed ledger or in the closing epoch (with its proof), from a maximum-depth tip that the latest closed ledger holds under a notarization lock, or from a complete current-epoch parent. The signed admission-witness digest fixes the anchor and old evidence actually used.

Why a recovery-only tip must be notarized again. An earlier version let a maximum-depth retained tip always be extended. That is unsafe. A recovery record of origin $e-1$ on B' is discharged by $\mathsf{XW}_{e-1}(B)$ for a rival B, so B may become final in epoch e on that witness, under the overlap exception or after installation. If a child C' of B' is then admitted without a new notarization of B', a fast certificate for C' finalizes B' as well (Case 5 of Appendix F). Lemma 1 compares supports at one position only, and C''s certificate puts no epoch-e vote at B's position, so nothing else excludes the pair. With the rule, finalizing C' needs $\mathsf{NC}_e(B')$, which $\mathsf{NC}_e(B)$ excludes; Lemma 2 states this in general.

Ordinary early work is provisional: when the predecessor arrives, it is rechecked against the decided frontier, finalized history, and carried lock records, and a tally of fast-final size does not bypass this check. A notarization lock constrains settlement to its branch. A recovery record of origin o on a block B' is *discharged* by a verified exclusion witness $\mathsf{XW}_o(B)$ for a block B that conflicts with B': account exclusion (Lemma 1) then proves that no epoch-o final certificate for B' can exist. The witness is retained. Evidence of any other epoch does not discharge the record, and a block carrying several records is released only when each has been discharged by a witness of its own origin.

The core overlap exception. A block may finalize before S_{e-1} is known only if the verifier checks its entire selected prefix against the latest closed state and proves cross-boundary exclusion at *every unresolved position being finalized*: each such block, including the certificate target, must have a closing-epoch exclusion witness XW_{e-1} and a current-epoch NC_e for the same hash. A closing-epoch finalized prefix is already a safe anchor. Any inherited recovery record bypassed by this path requires the matching-origin discharge witness above; inherited notarization locks cannot be bypassed. We call the bundle of these witnesses, the final certificate, and the discharge witnesses for the bypassed records an *overlap certificate* OC. Its validity is a function of its contents and of the last closed state alone (Lemma 8), so a validator that learns it before installing S_{e-1} and one that learns it after reach the same verdict.

A fresh descendant without its own closing-epoch exclusion witness remains provisional, even if its parent is notarized in both epochs; otherwise disjoint committees could certify different children of that parent. If the predecessor arrives first, nonfinal early work falls back to ordinary rechecking; already finalized overlap work remains in the certified live overlay.

Fresh-child recovery. If a checkpoint retains unresolved tips, the owner can extend a permissible tip with a fresh child; finalizing that child selects its unresolved ancestors rather than reopening their slots, and an undischarged lock record still constrains the branch choice. A tip held under a notarization lock is permissible at once; a tip held by recovery records alone becomes permissible in an epoch once it is notarized there again. Under the termination rules the owner need not act at all: the forked position itself becomes final within two good epochs (Theorem 4). Persistent submission therefore means retrying an omitted block or supplying a fresh, possibly empty, uncontested child of a retained admissible tip.

APPENDIX C
FORMAL EPOCH TRANSITIONS

A. Lagged committees and local boundaries

The account committee for epoch e is $K_e = C_{e-2}$. That outgoing committee decides S_e; the already known successor $C_{e-1} = K_{e+1}$ verifies and installs it. The state S_e derives C_e, whose account service starts in epoch $e+2$. Genesis supplies $S_{-1} = S_G$ and $C_{-2} = C_{-1} = C_G$.

After local duration D, a validator may end epoch e only when no older epoch remains closing. It stops old first and final voting, records all complete eligible old blocks as notarized, freezes its report against the closed S_{e-1}, and opens $e+1$; learning a decided checkpoint also stops any remaining signing for its closed epoch. A validator signs nothing new in an epoch it has left except late notarizations, which stop when it installs the checkpoint, but re-broadcasts what it signed before, so that certificates of the closing epoch can still complete. If closure outlasts D, the next local boundary is deferred while successor voting continues. Boundaries are local events; nothing requires a synchronized stop.

B. Immutable signed reports

Every correct outgoing validator i signs one header $R_i = \mathsf{Sign}_i(\mathsf{ctx}_e, \mathsf{Root}(T_i), \mathsf{Root}(G_i))$, where the context binds the session, closing epoch, predecessor checkpoint, outgoing committee, and reporter identity. A Byzantine identity may equivocate, but a candidate selects at most one of its headers. Let L_i be the reporter's ledger at freeze time. Its canonical tagged set is $T_i = \{(h(B), s_i(B)) : B \in L_i\}$ with $s_i(B) \in \{\mathsf{R}, \mathsf{N}, \mathsf{F}\}$: R is inherited recovery-protected state, N represented notarization without locally observed finality, F finality by inherited checkpoint state or by an explicit account certificate for the block or a descendant selecting it; the strongest status wins. Each entry also names the block's account position and parent, so that a selection places every block it names without the block body. Let $V_i^{(e)}$ contain the hashes of all blocks for which i released an epoch-e first vote before freezing. The second committed set is

$$G_i = V_i^{(e)} \setminus \{h : (h, \mathsf{N}) \in T_i \text{ or } (h, \mathsf{F}) \in T_i\}.$$

An N or F entry already exposes the vote. An R entry does not: it records what the predecessor carried, not what i signed in epoch e, so a reporter that first-voted an R-tagged block lists it in both T_i and G_i. An earlier version removed every key of T_i; Appendix E shows how that hides a fresh vote and lets two conflicting fast certificates form.

Correct reporters retain the frozen sets, vote records, and admission evidence; later events change their live view but not the signed report. Late notarizations are released after freezing, so they are in no $V_i^{(e)}$; they create no N entry, no Rule 1 or Rule 2 support, and no Rule 3 promotion.

C. Usable reports

Reconstruction is provided by the convergence service with integrity C1; RAI adds the semantic check. A report is *usable* at j when both sets are reconstructed, every entry is well formed, and its claims are justified by evidence j holds: an R entry matches predecessor recovery state; an N entry has the relevant NC of q first votes or an inherited notarization record; an F entry has valid inherited checkpoint finality, an explicit account certificate, or an overlap certificate; and a G_i membership is backed by i's own signed epoch-e first vote and names no hash that T_i tags N or F (it may name one tagged R). A report naming a block above the first position without its parent, unless the predecessor finalized that block there, is malformed and never selected, so one Byzantine report cannot make every candidate unconstructible. Missing evidence is fetched from retained data; a Byzantine report with unavailable or malformed material stays unusable. Closure needs only $N - f$ usable reports, and at least that many reporters are correct.

D. Eligible candidates

A candidate selects exactly $N - f$ usable report headers from distinct members of K_e, in canonical identity order, forming Q, and commits to an evidence manifest $\mathcal{M}$ listing content-addressed blocks, report reconstructions, the exact supporting vote records including the late notarizations inside exclusion witnesses, overlap certificates, and finite admission and dependency witnesses, all of them retained protocol data. The candidate value is $v_e = (Q, \mu_e, d_e)$ with $\mu_e = H(\mathcal{M})$, $d_e = H(S_e)$, and $S_e = \mathsf{BuildState}(S_{e-1}, Q, \mathcal{M})$. RAI's eligibility predicate, which the consensus service enforces through A2, holds for $v = (Q, \mu, d)$ when Q has $N - f$ usable reports of distinct members of K_e, $H(\mathcal{M}) = \mu$, $\mathsf{BuildState}(S_{e-1}, Q, \mathcal{M}) = S \neq \bot$, and $H(S) = d$, with all contexts, report identities, vote domains, and predecessor references verified. A correct member evaluates $\mathsf{Eligible}_e$ only on reconstructed and validated inputs; a digest establishes neither availability nor validity. The manifest fixes the witness bytes before voting, so the state digest is a function of fixed semantic inputs rather than of certificate arrival order; it is not a license to omit required report material, nor may it promote an N or G_i entry to final because an incidental certificate is present.

E. Deterministic construction

A candidate block is an inherited block, a selected tagged block, or a block named by a selected G_i, with the ancestors and fixed dependencies needed to validate it. Eligibility is checked against the closed predecessor or the overlap evidence retained with its admission, never against the candidate's own certificate, which would make Rule 2 circular; an unexplained speculative path is excluded from new recovery counts. For a block B, let $\mathsf{f}_Q(B)$ mean that a selected F entry names B and its finality witness verifies, and let the fresh reporter support set $U_Q(B)$ contain the selected reporters i with $h(B) \in G_i$ whose epoch-e first vote for B the manifest verifies. An inherited R entry is not itself support; an epoch-e first vote for an R-tagged block is, and reaches U_Q through G_i.

Final closure. Start with inherited finality and all blocks satisfying f_Q. Take account-ancestor closure and recursively include the explicit finalized-send dependencies required by those final blocks. Call the result $F^\star$.

Rule 1: represented notarization. At each new account position, after excluding conflicts with $F^\star$, retain the unique eligible represented NC target and its ancestry as a notarization lock; same-domain exclusion rules out two conflicting eligible NCs.

Rule 2: reporter first votes. If there is no represented eligible NC at that position and some eligible block has $|U_Q(B)| \geq 1$, retain the eligible block with the largest $|U_Q(B)|$, ties broken by the smaller hash, and its ancestry under a recovery record of origin e, the closing epoch, whether or not the block already carries records of earlier origins. A formable fast-final block has $|U_Q(B)| \geq r$ and every rival fewer, so it is the one retained (Theorem 1). An earlier version retained a block only if it alone reached r and otherwise nothing; a position the checkpoint leaves empty is voted afresh, and an adversary that can split the fresh votes can keep it open (Theorem 4).

Inherited state and discharge. Carry every predecessor lock record forward unless explicit compatible finality excludes its block or, for a recovery record of origin o, the manifest contains an exclusion witness $\mathsf{XW}_o(\cdot)$ for a conflicting block. A selected vote or claim for a block that conflicts with a carried record is eligible only if the manifest contains that witness; the record is then discharged in this construction. A block leaves the retained state only when none of its records survives and it is not an ancestor of a block with a surviving record. Discharge is never inferred from a certificate's newer epoch number, and all evidence summaries are computed before any omission. Let R_Q be the surviving inherited unresolved state, selected lock targets, and their ancestor closure.

Rule 3: checkpoint promotion. For each account, starting at the tip of $F^\star$, promote the next block only while it is the unique child retained in R_Q and the manifest verifies a closing-epoch NC of first votes for it; stop at the first fork or block without such an NC. The prefix P_Q is candidate-final until the consensus service decides the candidate. Recovery-only blocks are never promoted. Let F^+ be the ancestor- and finalized-send-dependency closure of $F^\star \cup P_Q$; the output ledger marks F^+ final and retains $R_Q \setminus F^+$.

Replay orders parents before children and finalized sends before dependent receives, choosing the least $(a, v, h(B))$ among independent ready blocks. A cyclic or semantically invalid graph is rejected; Lemma 9 shows that an all-correct usable selection never is.

F. Installing a decided checkpoint

Let a validator's current closed base be S_b with finalized set F_b, and O_i its certified overlay, so that its effective final set is $J_i = F_b \cup O_i$. On an accepted proof π_e for v_e, the validator recomputes S_e from the inputs v_e names, checks

Algorithm 1 BuildState$(S_{e-1}, Q, \mathcal{M})$

1: Verify the predecessor, selected identities, report roots, and the committed manifest.
2: Take every selected (T_i, G_i) from Reconstruct; verify the material for all required memberships and claims.
3: Form the parent-closed candidate set; classify admission and eligibility from fixed evidence.
4: Compute $F^\star$ by explicit-finality, ancestor, and finalized-send dependency closure.
5: Compute represented NC targets (first votes only) and support sets U_Q from every selected G_i, including members that T_i tags R, for every position before pruning.
6: For each inherited recovery record, look for an exclusion witness of the record's origin in $\mathcal{M}$ and discharge that record only. Verify each F claim that rests on an overlap certificate (Lemma 8). Reject claims that conflict with a surviving record.
7: Apply Rules 1–2; stamp each Rule 2 record with origin e; retain surviving inherited records and the ancestor closure of required lock targets, forming R_Q.
8: Apply Rule 3 to obtain P_Q; compute F^+ from $F^\star \cup P_Q$.
9: Retain $R_Q \setminus F^+$ and replay $F^+ \setminus F_{e-1}$ once in canonical dependency order.
10: Return (L_e, Σ_e); Rule 3 has effect only if this candidate is decided.

$H(S_e) = d_e$, and installs it with final set F_e: $J'_i = J_i \cup F_e$ and $O'_i = J'_i \setminus F_e$. The union is checked for account compatibility, final dependencies, and valid lock discharge; the validator adopts S_e as its base and materializes the live view as Replay(Σ_e, O'_i), with persistent operation identifiers suppressing duplicate application. The base's final set grows monotonically; every overlay block has an explicit account-finality proof for itself or a selecting descendant; and the union extends each base-final account prefix and respects every undischarged lock record.

Committee C_{e-1} holds no second election: it verifies the outgoing committee's proof, reconstructs the state, derives C_e from the canonical Σ_e, and rechecks nonfinal early work, discarding invalidated provisional work without clearing its persisted first vote or terminal state. Locks the checkpoint retains are continued, not reopened: every retained block gets an instance in the open epoch at its own position, a rival at that position is refused while any record on the retained block survives, and under the termination rules of Appendix B-B a rival whose witness discharges the records is admitted but not voted. A first vote in that instance is an ordinary epoch-e first vote; if the block does not finalize, the vote is exposed at the next closure through G_i, and Rule 2 gives the block a record of origin e. An NC_e assembled in that instance is also what lets a block held by recovery records alone serve as a parent in epoch e (Appendix B-C).

Appendix D
Full Proofs

The arguments concern the core protocol, including the prefix-wide overlap guard, late notarization, the early-vote rule, origin-stamped records, and Rule 3 promotion, and are relative to the services of Appendix A-D: safety uses C1 and A1–A3, termination additionally C2 and A4. Cryptographic guarantees hold except with negligible probability.

Lemma 1 (Account exclusion): In one account instance at most one block has an exclusion witness; in particular at most one has an NC. If an FC or FF for B can be formed in that instance, no conflicting block has an exclusion witness. If an FF for B can be formed, every conflicting block has at most $f + p < r$ supporters.

Proof: Two sets of q supporters intersect in at least $2q - N = f + 1$ identities, one of them correct, and a correct validator supports one block per instance (Definition 1), by a first vote or by a late notarization. An FC for B has q final voters; a correct final voter first-voted B in the same instance, so the same intersection applies against any XW of a rival. An FF has $N - p \geq q$ first voters. For the last claim let $f' \leq f$ be the actual number of Byzantine members. An FF has at least $N - p - f'$ correct first voters, so at most p correct members and f' Byzantine members can support a rival: at most $p + f' \leq p + f$. ∎

Lemma 2 (Notarized ancestry): If an epoch-e NC, FC, or FF for a block D can be formed, then every ancestor X of D is final, or is held under a notarization lock of a decided checkpoint (as its target or as an ancestor of it), or has an epoch-e NC that can be formed.

Proof: Each of the three certificates contains at least $q \geq f + 1$ first votes for D (a correct final voter first-voted the same block in the instance), so some correct validator first-voted D. By the attachment rule of Appendix B-C it did so on a parent P that it held final, held under a notarization lock, or held complete in epoch e, that is, with an $\mathsf{NC}_e(P)$ in hand. Ancestors of a final block are final, and ancestors of a notarization-lock target are retained under that lock, which Rule 1 creates with the target's ancestry and the inherited-state rule never drops while the target is unresolved. In the third case $\mathsf{NC}_e(P)$ can be formed and the argument repeats for P; the chain is finite. ∎

The lemma rests on the re-notarization rule of Appendix B-C: a tip held by recovery records alone is not a final parent. With the earlier rule, under which such a tip could always be extended, it fails, and so did Theorem 2 (Case 5 of Appendix F).

Proof structure. Theorems 1 and 2 are proved by one induction over closures, from genesis. The step for closure e uses Theorem 2 for finality through epoch e (in Rule 1's exclusion of conflicts with $F^\star$ and in the earlier-epochs case) and Theorem 1 for closure $e-1$ (the closure invariant of S_{e-1}); Theorem 2 up to epoch $e+1$ uses Theorem 1 for closures up to e. Each proof below cites the other only within this order.

Lemma 3 (Late notarization adds no old finality): Every epoch-e NC, FC, or FF consists of first and final votes released by correct signers before they left epoch e, together with Byzantine votes. A correct validator releases a final vote, or records an N entry, only for a block for which it holds an NC of q first votes.

Proof: By Definition 1 and the certificate definitions of Appendix B-B none of the three certificates counts a late

notarization, and a correct validator releases no epoch-e first or final vote after leaving e (Appendix C-A). Completeness is defined on NC_e, and notarization and final voting require completeness. ∎

Lemma 4 (Frozen-report exposure): Let i be a correct reporter for closure e. If i released an epoch-e final vote for B, then T_i tags B with N or F. If i released an epoch-e first vote for B, then T_i tags B with N or F, or $h(B) \in G_i$. This holds whatever other tag B inherits from S_{e-1}, and it persists after the report is frozen.

Proof: A final vote follows durable notarization, which is an N entry unless finality has replaced it by F. For a first vote, $h(B) \in V_i^{(e)}$, and G_i removes from $V_i^{(e)}$ exactly the hashes tagged N or F. Stopping precedes freezing; roots and tags are then immutable, and C1 ensures that what a correct validator reconstructs is what was frozen. ∎

Remark 1: With the earlier definition $G_i = V_i^{(e)} \setminus \mathsf{keys}(T_i)$ the second statement fails when S_{e-1} carries B under a recovery record: T_i tags B with R and the fresh vote appears nowhere. `RaiClose` with `FIXB = FALSE` reaches two conflicting fast certificates this way (Appendix E).

Lemma 5 (Signers hold the witness): Every correct signer of an eligible epoch-e FC or FF for B holds, as retained evidence, an exclusion witness discharging each recovery record of S_{e-1} on a block that conflicts with B.

Proof: An $\mathsf{FF}_e(B)$ consists of settled first votes. A correct signer of one had installed S_{e-1} when signing; the voting rule of Appendix C-F refuses a rival of a retained block unless every record on it is discharged, and the signer's admission witness fixes the witnesses it used, so it holds them. A correct final voter either released its final vote after installing S_{e-1}, with the same check, or before. In the second case the early-vote rule required it to hold $\mathsf{XW}_{e-1}(B)$ for B and every unresolved block of the selected prefix. That witness discharges every record on a rival that closure $e-1$ created, since those have origin $e-1$ (Rule 2 stamps the closing epoch). Records that S_{e-1} inherits from S_{e-2} were known to the voter, because S_{e-2} is installed before the overlap of closure $e-1$ begins (at most one closing epoch per validator), and the overlap guard required the matching-origin witness for each of them. ∎

Definition 2 (Closure invariant): A closed state S_e satisfies the closure invariant if, for every epoch $e' \le e$ and every block B with an eligible epoch-e' account final certificate that can be formed from released signatures, S_e holds B (i) as final, (ii) under a notarization lock, or (iii) under a recovery record of origin e', directly or as an ancestor of a block so held.

Theorem 1 (Closure preserves latent and late finality): Let S_{e-1} be a valid closed predecessor that satisfies the closure invariant, and let v be any candidate with $\mathsf{Eligible}_e(v)$ and state S_e. Let B have an eligible epoch-e account final certificate that can be formed, possibly completed after reports froze or after the checkpoint was decided, and assume

(W) B conflicts with no lock record carried by S_{e-1}, or every correct signer of the certificate holds, as retained evidence, an exclusion witness that discharges each such record.

Under the early-vote rule, (W) holds for every eligible certificate (Lemma 5); without that rule it can fail (Appendix D-A). Then S_e holds B as final, under a notarization lock, or under a recovery record of origin e, directly or as an ancestor of a block so held; no admissible evidence discharges that hold; and S_e satisfies the closure invariant for all certificates covered by (W). The same holds for every ancestor of B that the certificate finalizes with it.

Proof: Let Q be the selected reporters, $|Q| = N - f$, and let A be the signer set of the certificate.

Eligibility of B in the construction. If B conflicts with no carried record there is nothing to show. Otherwise, by (W), each correct signer holds an exclusion witness for every carried record that B bypasses. That witness is retained evidence; reconstructing the signer's report material delivers it, the manifest contains it, and the record is discharged at line 6 of Algorithm 1. So B is eligible for Rules 1 and 2 whenever a correct signer is selected, which the counts below guarantee.

Normal certificate. A is a set of q final voters, so $|(Q \cap A) \setminus \mathsf{Byz}| \ge (N-f) + q - N - f = q - 2f = p + 1 \ge 1$. By Lemma 4 each such reporter froze an N or F entry for B. A verified F entry puts B in $F^\star$. Otherwise B is a represented NC target; it is the unique one at its position by Lemma 1, it does not conflict with $F^\star$ by the induction hypothesis of Theorem 2, and Rule 1 retains it under a notarization lock.

Fast certificate. A is a set of $N - p$ settled first voters, so $|(Q \cap A) \setminus \mathsf{Byz}| \ge (N-f)+(N-p)-N-f = N-p-2f = r$. Take these reporters. If any selected report has a verified F entry for B we are in the first case. Else if any has a verified N entry for B, Rule 1 applies as above. Else, by Lemma 4, each of the r reporters has $h(B) \in G_i$ with a verified first vote, whether or not T_i tags B with R; hence $|U_Q(B)| \ge r$. No rival has a represented NC, and every rival B' has $|U_Q(B')| \le f + p < r$, both by Lemma 1; so B is the unique eligible block with the most reporter votes, and Rule 2 retains it under a recovery record of origin e.

No discharge. A notarization lock is never bypassed. A record of origin e on B is discharged only by $\mathsf{XW}_e(B')$ for a conflicting B', which Lemma 1 excludes because an epoch-e final certificate for B can be formed. Evidence of other epochs does not apply to it, and discharging records of other origins on B leaves it in place.

Finality through a descendant. Let X be an ancestor of B that is not final before the certificate. If S_{e-1} holds X final or under a notarization lock, the inherited-state rule keeps it. Otherwise S_e holds X as an ancestor of B, by the cases above, and the question is whether that hold can be discharged. A hold through B ends only when every record on B is discharged or a final block excludes B. A record of origin e on B is discharged only by $\mathsf{XW}_e(Y)$ for the block Y of a conflicting branch at the first position where that branch leaves B's; that position holds B or an ancestor X of B. By Lemma 2, X is final, under a notarization lock, or has an $\mathsf{NC}_e(X)$ that can be formed; the first two exclude the conflicting branch outright, and in the third Lemma 1 excludes $\mathsf{XW}_e(Y)$. A notarization lock on B or finality of B is never undone. This is the step

the earlier retained-tip exemption broke: without Lemma 2, an $\mathsf{XW}_e(Y)$ can exist at an ancestor's position, and in Case 5 of Appendix F Y even becomes final.

Earlier epochs. For $e' < e$ the predecessor holds B as final, under a notarization lock, or under a record of origin e'. Finality is inherited. A notarization lock and a record of origin e' are carried forward by the inherited-state rule, since the only ways to drop them are a conflicting final block, excluded by Theorem 2, and an $\mathsf{XW}_{e'}$ of a rival, excluded by Lemma 1.

The argument uses only signatures in A and their intersection with Q, so it does not matter when the certificate is assembled, and unselected or Byzantine signatures cannot remove the correct intersections. ∎

Lemma 6 (Checkpoint-promotion safety): A Rule 3 promoted block cannot conflict with a block that has a closing-epoch exclusion witness, FC, or FF. Hence a decision that finalizes the promoted prefix is compatible with old-domain finality and with core overlap finality.

Proof: Every promoted block has a verified closing-epoch NC of first votes, which is an exclusion witness; Lemma 1 excludes an exclusion witness, and so an FC or FF, for a conflicting block. Core overlap finality for a conflicting successor block requires a closing-epoch exclusion witness at the first divergent position, which is the same contradiction. Promotion stops at retained forks and never promotes recovery-only evidence. ∎

Lemma 7 (Acyclic final dependencies): For any finite set of valid final blocks closed under account ancestors and required finalized sends, parent and send-to-receive edges form a directed acyclic graph.

Proof: Order blocks by the earliest event at which valid final evidence justifies them. Along a parent edge this event cannot become earlier at the child; along a send-to-receive edge it is strictly later at the receive, because admission required the send's finality proof and Rule 3 cannot supply it for the checkpoint under construction. A cycle cannot consist only of parent edges, since slots increase strictly, so it contains a send-to-receive edge and a strict increase around a cycle, a contradiction. ∎

Lemma 8 (The overlap verdict does not depend on installation order): Let closure e be in progress, let B and B' conflict at one position, and let OC be an overlap certificate for B: an exclusion witness $\mathsf{XW}_e(B)$, an $\mathsf{NC}_{e+1}(B)$, an epoch-$(e+1)$ FC or FF for B or a descendant selecting it, and an exclusion witness of the matching origin for every recovery record on B' that S_{e-1} carries, with no notarization lock on B' in S_{e-1}. Then for every candidate with $\mathsf{Eligible}_e$ and state S_e: (a) S_e does not finalize B'; (b) S_e carries no notarization lock on B'; (c) every recovery record that S_e carries on B' is discharged by a witness contained in OC. Consequently a validator that holds OC accepts B as final under the overlap exception before installing S_e if and only if it accepts B by ordinary rechecking after installing S_e, and a later manifest that contains OC verifies an F claim for B against S_e.

Proof: (a) S_e finalizes a block through inherited finality, an explicit closing-epoch account certificate, or Rule 3. Inherited finality of B' contradicts the check of OC against S_{e-1}, which is installed before the overlap of closure e begins. An epoch-e FC or FF for B', and the $\mathsf{NC}_e(B')$ that Rule 3 requires, are excluded by $\mathsf{XW}_e(B)$ and Lemma 1. (b) A new notarization lock needs a represented $\mathsf{NC}_e(B')$, excluded the same way; an inherited one is excluded by hypothesis. (c) A record on B' in S_e is either created by Rule 2 at this closure, with origin e, and then $\mathsf{XW}_e(B) \subseteq \mathsf{OC}$ discharges it; or inherited from S_{e-1}, and then OC contains its discharge witness by hypothesis.

For the consequence: before installation the validator checks exactly the contents of OC against S_{e-1}. After installation, ordinary rechecking tests B against the frontier, the finalized history, and the carried records of S_e; by (a)–(c) these tests pass on the evidence in OC and on no other. The same evaluation is what line 6 of Algorithm 1 performs at closure $e+1$. The lemma is stated for one divergent position; the prefix-wide guard applies it at the first divergent unresolved position of the finalized prefix, as the proof of Theorem 2 does, and the `RaiPrefix` model of Appendix E checks that guard on two positions. ∎

Lemma 9 (Deterministic construction): Assume the predecessor satisfies the closure invariant. For fixed usable selected reports and a valid immutable manifest, BuildState returns one canonical candidate state at every correct validator. It preserves inherited finality, carries all mandatory surviving records, and applies only explicit account finality plus Rule 3 promotion. All-correct usable inputs yield a finite valid construction.

Proof: By C1 every correct validator reconstructs the same (T_i, G_i) for each selected header, and the manifest fixes witnesses and proof choices; hence explicit finality, discharges, and R_Q are the same everywhere, with lock records compared as sets of (block, strength, origin). Lemma 1, predecessor validity, attachment, and Theorem 1 make the retained finality and records compatible. Rule 3's tests fix P_Q, safe by Lemma 6; Lemma 7 and the canonical tie-break fix replay. An N entry of a correct reporter is backed by an NC of first votes (Lemma 3), which the manifest can carry, so the entry verifies. An F entry of a correct reporter for a block finalized under the overlap exception is backed by its overlap certificate, which is retained evidence; by Lemma 8 line 6 accepts the claim and discharges every conflicting record of S_{e-1}, so no all-correct selection is rejected on that account. Malformed Byzantine inputs are excluded by usability; all-correct usable inputs yield a finite state. ∎

Theorem 2 (Transfer safety across epochs): Assuming C1 and A1–A3, finalized histories at correct validators are compatible and irreversible across all core-protocol epochs; finalized balances are nonnegative, and each send is received at most once.

Proof: Induct from genesis, jointly with Theorem 1. *Within an epoch*, let two blocks be final through epoch-e certificates for D and D' whose branches conflict, and let $X \neq X'$ be their blocks at the first position where the branches differ. Conflicting finality need not be visible at a single position: either certificate may finalize X or X' only as

an ancestor. By Lemma 2 each of X, X' is final through older evidence, held under a notarization lock, or has an NC_e that can be formed. Two NC_e at one position contradict Lemma 1. A notarization lock on X is never bypassed: a correct validator that installed it refuses X' and its descendants, and one that had not accepts D' only under an overlap certificate, whose $\mathsf{XW}_{e-1}(X')$ contradicts the closing-epoch NC behind a new lock (Lemma 1) and whose check against the latest closed state refuses an inherited one. Finality through older evidence is the boundary case. At a boundary, A2 ensures that the decided v_e is eligible, so by Theorem 1 it preserves every formable old final branch and by Lemma 6 its promotion is compatible; A1 and A3 ensure that every correct validator installs the state of the same v_e, which by Lemma 9 is the same state. For overlap, the first divergent unresolved position on an early successor-final prefix has a closing-epoch exclusion witness for the successor block by the prefix-wide guard, whose late notarizations add no old finality that Theorem 1 would have to preserve (Lemma 3); a conflicting closing-epoch certificate or a conflicting Rule 3 promotion would require a conflicting NC, contradicting Lemma 1; and by Lemma 8 correct validators that hold the same overlap certificate reach the same verdict whether they verify it before or after installing S_e, so they agree on which overlap blocks are final and not only on the compatibility of what they have finalized. Matching-origin discharge and inherited records remain enforced after installation, and the base/overlay union with applied-operation identifiers prevents rollback and duplicate application. Replay checks balances and finalized-send dependencies. ∎

Lemma 10 (Correct reports become usable): Under the data service, finite relevant admitted work, fair processing, and C2, every correct report eventually becomes usable at every correct validator.

Proof: A correct reporter stops old first and final voting, so its old vote set and roots are finite, and its report is well formed: every N entry rests on an NC of first votes (Lemma 3), every F entry on inherited finality, an account certificate, or a retained overlap certificate, and every G_i member on its own first vote and on no hash tagged N or F. By C2 every correct validator eventually reconstructs (T_i, G_i). The data service delivers the blocks, ancestry, signed votes, late notarizations, and overlap certificates behind its claims, and fair processing completes the finite validation. A withholding Byzantine reporter can make only its own report unusable; $N - f$ correct reports remain. ∎

Theorem 3 (Handoff termination): Assume the data service, fair processing, C2, A4, and recurring prepared windows with $N - f$ responsive correct members of K_e long enough for reconstruction, candidate preparation, and the consensus service's rounds. Every enabled handoff is eventually decided and, with eventual retained-data delivery, installed by correct successors.

Proof: By Lemma 10, $N - f$ correct reports become usable at every correct member, which by Lemma 9 can then construct an eligible candidate; the canonical selection order makes correct members that hold the same usable reports construct the same one. A4 then decides some eligible value in a sufficient window; A3 lets successors verify it, and retained authorizing data let them recompute and install the state, which enables the next deferred boundary. ∎

Theorem 4 (Account continuation and forked positions): Assume handoff termination, available final-send dependencies, and recurring common-open ready intervals. (a) An owner following the retry-or-fresh-child policy of Appendix B eventually finalizes each target of a finite valid uncontested continuation, provided each required new position has such an interval, and the finalized targets eventually enter a checkpoint. (b) Let (a, v) be a position whose parent is final when epoch e starts and at which some valid block is available. If epochs e and $e+1$ are *good*, some block at (a, v) is final by the end of epoch $e+1$, whatever the owner and the Byzantine validators do. An epoch is good if every correct member of its committee installs the predecessor checkpoint, receives every block and vote that a correct validator sends at the position, and casts every first and final vote the rules allow it before freezing its report. (c) Hence the owner of a forked account, once it signs no further conflicting blocks, finalizes its continuation as in (a) after the forked position is final; and at most one block at the position ever becomes final.

Proof: (a) In a ready interval, $N - f \geq q$ correct first votes create an NC and the same validators can final-vote; after they install the predecessor their first votes are settled, so an available $N-p$ fast quorum finalizes in two stages. An attempt interrupted by a boundary is preserved by Theorem 1; an omitted block can be retried, and a uniquely retained notarized target may finalize at the handoff.

(b) Consider epoch e. Every correct member installs S_{e-1} and first-votes at the position before freezing; where the position is contested for it, it votes only after installing S_{e-1} and then, if S_{e-1} holds an unresolved block K there, only K. Suppose no block at the position is final at the end of e. Then the position is contested at every correct member: if S_{e-1} holds a block there, every correct member installed it; otherwise, if all correct members first-voted the same block B, then B had $N-f \geq q$ correct first votes, every correct member final-voted it after installing S_{e-1} (nothing there refuses it), and B would be final; so correct members voted two blocks, and each has seen both.

Next, S_e holds an unresolved block K' at the position or finalizes one. If a selected report represents an eligible NC_e, Rule 1 locks it, and Rule 3 promotes it unless another block stays retained there. Otherwise, a record of S_{e-1} that survives keeps its block; and if none survives, every block at the position is eligible, the selected reports contain at least $N-2f \geq 1$ correct reports, each listing its member's epoch-e first vote unless it tags it N or F, and Rule 2 retains the block with the most reporter votes.

Now consider epoch $e+1$. The position is contested at every correct member, so each first-votes there only after installing S_e, and then only K', unless a rival is final, which would end the argument. The cross-epoch rule does not interfere, since it lapses at installation. K' is retained, so every correct

member admits its instance, and in a good epoch it receives $N - f \geq q$ correct first votes. The same members final-vote K' after installation, which the termination rules allow because K' is the block S_e keeps, and q correct final votes make it final. A block whose parent position is final needs no complete parent, and the re-notarization rule concerns only children of K'.

(c) Once a block at the forked position is final, the account's later positions are uncontested if the owner signs no further conflicting blocks, and (a) applies; a fresh child of the final block is the owner's continuation. At most one block at the position becomes final by Theorem 2.

The rules of (b) only withhold votes, except Rule 2's choice, which retains every block that may be fast-final (Theorem 1); so they do not weaken safety. Without them the position can stay open forever (Appendix D-B). Normal finality takes three delivery stages after broadcast and an $N - p$ settled first-vote quorum two; a sufficient account interval budget is $D/(1+\rho) > \kappa_A + R_A + 3\delta$, where κ_A bounds opening skew and R_A readiness after the last opening, including predecessor closure. ∎

A. Early votes against a carried lock

Without the early-vote rule, hypothesis (W) of Theorem 1 is not cosmetic. The following run violates the theorem in the protocol as it stood before the repairs, and equally after Fix B alone. `RaiHandoff` finds it with `RULE = FALSE`. Take $N = 6$ with validator 6 Byzantine, and conflicting blocks B, B'.

1) Epoch e: validators 1, 2, 6 first-vote B'. Validators 3, 4, 5 cross their boundary without voting at this position.
2) Before S_e is known, 3, 4, 5 first-vote B in epoch $e+1$ and late-notarize it in e. Validator 6 signs both votes for B and shows its late notarization to validator 1 only.
3) Closure e selects reports 1, 2, 3, 4, 6. Rule 2 gives B' a recovery record of origin e.
4) Validators 3, 4, 5 install S_e without holding $\mathsf{XW}_e(B)$; their early work on B is invalid in their view. Validator 1 installs S_e, holds $\mathsf{XW}_e(B) = \{3,4,5,6\}$, discharges the record and first-votes B in $e+1$. Now $\{1,3,4,5,6\}$ is an $\mathsf{FF}_{e+1}(B)$ that can be formed.
5) Closure $e+1$ selects reports 2, 3, 4, 5, 6. The only correct holder of the witness is not selected, the manifest has no $\mathsf{XW}_e(B)$, the votes for B are an unexplained speculative path, and S_{e+1} retains B' only.
6) In $e+2$ validators 2, 3, 4, 5, 6 first-vote the retained B': $\mathsf{FF}_{e+2}(B')$. Validator 6 then publishes its signatures; B has an FF_{e+1} whose eligibility against S_e follows from $\mathsf{XW}_e(B)$.

With a committee change the same failure needs no late notarization: four new members early-vote B, the old committee's first votes for B plus one withheld Byzantine vote are the exclusion witness, and the second closure sees four unexplained votes. The cause is that the eligibility of an early first vote can depend on a witness its signer never held. The early-vote rule removes the dependency: in step 4 the set $\{1,3,4,5,6\}$ is no longer a fast certificate, because three of its votes are early, and validators 3, 4, 5 final-vote B only once they hold $\mathsf{XW}_e(B)$, in which case their reports deliver it to the next closure.

Two alternatives were considered and set aside. Retaining both blocks at the second closure keeps the one-round path but can stall a contested position permanently after a committee change. Counting only early votes of members of the closing committee is unsafe: the committee-change run above defeats it.

B. Why forked positions need the termination rules

Without the termination rules of Appendix B-B and the new Rule 2, a forked position can stay open forever although every epoch is good. Take the first run of Case 5 with the re-notarization rule, which refuses the owner's child C' until B' is notarized again. In each later epoch validators 3, 4, 5 first-vote B again: before installing the checkpoint the cross-epoch rule allows re-voting the same block, and after it the old witness $\mathsf{XW}_e(B)$ still admits B. Validators 1 and 2 vote B', and validator 6 withholds the one vote either side needs. Neither block is notarized, the next checkpoint keeps B' again, and the owner cannot help: C' waits for B'. Nor could a different child rule help: in the second run B may already be final when the owner publishes C', and validators 1 and 2 cannot tell the two runs apart. The earlier promise that the owner recovers by extending the kept block held only because that route was unsafe.

Each rule removes one ingredient. With *checkpoints decide*, validators 3, 4, 5 stop voting B after installing the checkpoint; with *no early votes at a contested position*, they do not re-vote it before; and with Rule 2 keeping the block with the most reporter votes, a checkpoint never leaves a voted position empty, which an adversary could otherwise split again. `RaiResolve` (Appendix E) finds the stall with the rules off, also when the position starts uncontested, and checks with them on that two good epochs finalize the position.

APPENDIX E
THE TLA$^+$ MODELS

The six modules, their TLC configurations, a run script, and the TLC outputs (including the violating traces) are in the `tla/` directory of the artifact. `RaiClose`, `RaiOverlap`, and `RaiHandoff` take one account position with two conflicting blocks X and Y, $N = 6$, $f = p = 1$, five correct validators and one Byzantine validator whose signatures are available for every block, vote kind, and epoch and whose report is chosen adversarially subject to the usability checks of Appendix C-C. `RaiPrefix` and `RaiRecovery` take two positions and are run with $N = 4$ ($f = 1$, $p = 0$) and $N = 6$. `RaiResolve` takes one position over three epochs and is run with $N = 4$. The consensus service is the nondeterministic choice of one eligible candidate from any $N - f$ usable reports. TLC explores all interleavings; a `VIEW` identifies validators up

to permutation. The runs of Table II used TLC build 2026-10-06 (two workers) and TLC 2.20 (eight workers, `RaiPrefix`).

Invariant names. `NoConflict` and `Agreement`: no two conflicting blocks are final (Theorem 2 at the position). `ClosureThm1`, `ClosureThm1a`, `ClosureThm1b`: Theorem 1 for the respective closure, stated without hypothesis (W); the archived outputs of the runs made before this revision name them `Thm64`, `Thm64a`, `Thm64b`. `OneFinal`: a checkpoint finalizes at most one block per position. `SameVerdict`: Lemma 8. `UsableReports`: Lemma 10 once every validator has left the epoch. `WitnessAdmissible`: line 6 of Algorithm 1 at the next closure can verify a correct F claim against an inherited lock on its rival. `EarlyCompatible` (`RaiPrefix`): every block finalized early is compatible with every formable epoch-1 certificate. In `RaiRecovery`, `NoConflict` counts a parent as final when its child is. `Probe*`: reachability probes, expected to be violated, which show that the invariants are not vacuous.

`RaiClose`: three voting epochs, two closures. Votes are cast only after the voter's predecessor checkpoint is decided, so there is no early voting, late notarization, or overlap finality. Each closure runs BuildState with Rules 1–3, inherited records with origins, and matching-origin discharge; the manifest's discharge set is any subset of the dischargeable records that contains those a selected correct vote relied on. Switch `FIXB` selects the definition of G_i. Abstractions, each of which only adds behaviours: a validator's activity in different epochs is carried by separate identities, so a validator may still sign in epoch e while epoch $e+1$ votes; a validator that observes its NC releases its final vote in the same step; reports are forgotten once their closure is decided.

`RaiOverlap`: closing epoch, open epoch, one closure. Validators cross their local boundary one by one. After it, and before installing S_1, a validator may first-vote in epoch 2 under the cross-epoch guard and then also signs a late notarization in epoch 1. A block may finalize before S_1 is known under the overlap exception, or by rechecking against S_1 after installation. Switches: `NCSPLIT` (a validator records N and final-votes only on an NC of q first votes; off, late notarizations count too), `DISCHARGE` (a recovery record is discharged by a conflicting epoch-1 exclusion witness; off, never), `LATEEV` (late notarizations are admissible manifest evidence for discharge; off, they are "never construction input"). Finality decisions are recorded globally rather than per validator: `ovf` for the overlap exception at a validator that has not installed S_1, `pof` for rechecking at one that has. Closure 2 is not built; its acceptance condition for the case of interest is `WitnessAdmissible`.

`RaiHandoff`: early voting, two closures, changing committee. Epoch 1 is voted and closed by K_1, epoch 2 by K_2; the Byzantine validator sits in both. A member of K_2 may first-vote in epoch 2 before installing S_1 (an early vote); if it is also in K_1 and supported nothing else in epoch 1, that vote comes with a late notarization in epoch 1. Fix B, the NC split, matching-origin discharge, and late votes as evidence are built in. `RULE = FALSE` is the earlier certificate definition: a fast certificate counts all first votes and a validator final-votes on its epoch-2 NC alone. `RULE = TRUE` is the early-vote rule: a fast certificate counts only first votes cast after the signer installed S_1, and an early final vote is released only by a validator that holds the epoch-1 exclusion witness. A block is final when its certificate can be formed from released signatures and it is valid against S_1, through the overlap bundle or by rechecking with discharge. The builder of S_2 holds a discharge witness only if a selected correct reporter relied on it; otherwise the adversary decides, which models a withheld signature. Further abstractions: correct reports carry no F tag for an observed certificate (covered by the other two models); a claim that conflicts with a surviving record is ignored rather than invalidating the selection, which only adds checkpoints; epoch 3 is not modelled, so safety after S_2 is checked as Theorem 1 for S_2.

`RaiPrefix`: the prefix-wide guard on two positions. One account with a parent P and a rival P' at position 1 and two children C_1, C_2 of P at position 2. Epoch 1 is closing and voted by K_1, epoch 2 is open and voted by K_2; one Byzantine validator sits in both committees, and the two committees are identical, disjoint, or share k members. Members of K_1 cross their boundary one by one. S_1 is never installed within the model, so every epoch-2 first vote is early: early votes form no fast certificate, and an epoch-2 final vote is released only under the overlap guard. A member of both committees that first-votes x in epoch 2 also signs a late epoch-1 notarization for x, unless it first-voted at that position in epoch 1, and the cross-epoch rule stops it from first-voting a different block where it supported one in epoch 1. The closure and BuildState are left out: early finality is checked against latent epoch-1 finality, that is, against any formable epoch-1 FC or FF. Switches: `GUARD = "prefix"` requires XW_1 and NC_2 for every unresolved block on the path to the anchor, `GUARD = "target"` checks only the finalizing block; `FRESHCHILD = TRUE` requires a child to have its own XW_1, `FALSE` lets NC_2 for the child plus a parent with NC_1 and NC_2 suffice. Abstractions that only add behaviours: votes are signer sets, checks read formable certificates, finality is global, and members of $K_2 \setminus K_1$ may vote in epoch 2 from the start.

`RaiRecovery`: building on a recovery-locked block. One account with conflicting blocks X and Y at position 1 and a child of each at position 2, C_X on X and C_Y on Y. Epoch 1 is voted by K_1 at position 1 and closed as in `RaiHandoff`; epoch 2 is voted by K_2 at both positions, early (before installing S_1, with late notarization, the cross-epoch rule, and the early-vote rule) or settled. Before installation a child is first-voted only on a parent complete in epoch 2 or final in epoch 1; after installation, on a parent final in S_1, or complete in epoch 2 and admissible against S_1, or, with `EXEMPT = TRUE`, retained by S_1 (the earlier retained-tip exemption). Early final votes need the prefix-wide overlap bundle with a fresh child witness. A final child finalizes its parent, and `NoConflict` counts that. Abstractions: nobody

votes at position 2 in epoch 1, finality is global, and closure 2 is not built, so the model checks Theorem 2 within epoch 2 and against S_1. Run with $N = 4$ and $N = 6$.

`RaiResolve`: termination at a forked position. One account position with conflicting blocks X and Y, three voting epochs, two closures, one committee of $N = 4$ ($f = 1$, $p = 0$). Voting follows Appendix B: one first vote per epoch, the cross-epoch rule, admission with matching-origin discharge, early final votes only with the closing-epoch witness, settled votes only on the fast path; a validator leaves epoch e only with S_{e-1} installed. The Byzantine validator signs first votes for either or both blocks in any epoch, at any time, or withholds them, reports adversarially within the usability checks, and the manifest discharges any dischargeable subset. Validators see the two blocks when the adversary delivers them. Each epoch is flagged good or not; in a good epoch no correct validator leaves before every correct validator has installed the predecessor, seen both blocks, and cast every first and final vote the rules allow. `TwoGood`: two consecutive good epochs make some block final. Switch `RULES`: the two termination rules and the new Rule 2. `Spec23` starts after an arbitrary fork epoch (any epoch-1 votes and Byzantine signatures, anything seen, any checkpoint closure 1 can build, installed by nobody yet) and makes epochs 2 and 3 good; `Spec` explores every pattern of good epochs and is run by simulation. Abstractions: no late notarizations and no committee change; a formable final certificate counts as finality, whose validity the safety models check; the Byzantine validator signs no final votes, which could only complete a certificate; a correct report tags a block N only after its final vote, which can only hide notarizations.

Rule 2 keeping a side (`MAXKEEP`). `RaiClose`, `RaiOverlap`, `RaiHandoff`, and `RaiRecovery` take a switch `MAXKEEP` for the new Rule 2; the archived configurations set it off.

Table II lists every configuration. The `Overlap_noSplit` run shows that counting late notarizations toward an NC damages usability, hence termination, not safety: with the split off the other four invariants hold over the full state space. The `Prefix` runs show that the two shortcuts one might take in the overlap rule are unsafe as soon as the committees are not identical: checking only the finalizing block (`target`) fails with two shared members at $N = 4$ and with three at $N = 6$, and letting a child inherit its parent's witness (`nofresh`) fails with disjoint committees and with one or two shared members; the prefix-wide guard with a fresh witness per child (`fixed`) passes every $N = 4$ configuration. The `Handoff` runs with one and four shared members, and the exhaustive `Prefix6` runs of the repaired guard, had not finished when this archive was made. The `Recovery` runs find the retained-tip exemption unsafe for every pair of committees at $N = 4$ and by simulation at $N = 6$, and the re-notarization rule safe in every $N = 4$ configuration and, at $N = 6$, in every run of up to 24 steps (the search was stopped at depth 25); the three probes show that the precondition (a final block against a recovery lock on its rival) and owner recovery, also after a validator voted the dropped side, are reachable. No model covers more than two positions, a third epoch, receive dependencies, restarts, other committees, or any N other than 4 and 6, and `RaiRecovery` builds no second checkpoint, so Theorem 1 with finality through a descendant is argued, not model-checked. The `Resolve` runs show the stall without the termination rules, from the start and from an arbitrary fork epoch, and with them that two good epochs after any fork epoch finalize the position, exhaustively for $N = 4$ and three epochs, and in every pattern of good epochs that half a million random runs reached; one good epoch is not enough. The new Rule 2 passes every safety configuration it was added to.

Appendix F
Scripted Regression Cases

The random Byzantine voter of the benchmark harness does not find the runs of Appendix E. The following scripted cases were derived from the violating traces; $N = 6$, validator 6 is Byzantine, B and B' conflict.

Case 1 (Fix B). Epoch 1: validators 1, 2, 3 vote B'; 4, 5 vote B; 6 reports B only; 3's report is not selected. Epoch 2: 1 to 4 first-vote the retained B, nobody assembles the NC. Release the epoch-1 NC for B' after the second close. Expected: the second checkpoint carries B with an origin-2 record; B finalizes; B' never does.

Case 2 (discharge). Before the boundary 1, 2, 6 vote B'. After it 3, 4, 5 vote B in the open epoch and late-notarize it; 6 joins both. Install the checkpoint at 1 and 2 before they see B's certificate. Expected: every validator finalizes B, and the next closure builds.

Mixed NC. One validator still in the closing epoch receives three first votes and one late notarization for its block. Expected: it does not record N; its report is usable.

Case 3 (early votes). The run of Appendix D-A. Expected: B has no fast certificate; 3, 4, 5 final-vote B only if they hold the witness, and then the second checkpoint finalizes B. Without the early-vote rule this run ends with both blocks final.

Case 3, new members. Epoch 1: 1, 2, 3 vote B; 4, 5 vote B'; 6 reports B' only and withholds its vote for B. Four members who were not in the old committee early-vote B. Same expectation as Case 3.

Case 5 (re-notarization). Epoch e: 1, 2, 6 first-vote B' at position 7; 3, 4, 5 cross their boundary without voting there. Before S_e is known, 3, 4, 5 first-vote B in epoch $e + 1$ and late-notarize it in e; 6 signs both for B, so $\mathsf{XW}_e(B)$ and $\mathsf{NC}_{e+1}(B)$ hold. Closure e selects 1, 2, 3, 4, 6 and keeps B' under a recovery record of origin e. 3, 4, 5 final-vote B, either after installing S_e, where $\mathsf{XW}_e(B)$ discharges the record, or before it under the early-vote rule once 6 shows them its signatures; with 6, B is final. The owner publishes C' on B' at position 8, and 1 to 5 first-vote it after installing S_e: a fast certificate for C' finalizes B' too. `RaiRecovery` with `EXEMPT = TRUE` finds a

shorter run without early votes: in epoch e, 1 and 2 vote X and 3 votes Y ($N = 4$), the checkpoint keeps Y on the Byzantine report, and after installing it 1, 2, 3 first-vote both X (whose witness discharges Y's record) and C_Y. Expected under the re-notarization rule: C' is refused at its first vote, because B' has at most the epoch-$(e+1)$ votes of 1, 2, and 6 at position 7; in a run where nobody voted B, B' is notarized again and C' finalizes both.

Case 6 (termination). The stall of Appendix D-B: after the checkpoint keeps B', validators 3, 4, 5 re-vote B before installing it and vote B again after installing it on its old witness; validator 6 withholds. Expected: under the termination rules, 3, 4, 5 do not first-vote at the contested position before installing the checkpoint and vote only B' after it, which is notarized and final in the next good epoch; with a three-to-three split in the reports the checkpoint keeps the side with the smaller hash.

Case 4 (prefix). Epoch 1: P is notarized; C_1 has first votes from 1, 2, 3. After the boundary, members of a committee sharing two members with the old one first-vote C_2 early and late-notarize it. Expected: C_2 does not finalize early without its own epoch-1 witness, and no validator finalizes C_2 while an epoch-1 certificate for C_1 can form.

`RaiClose` with `FIXB = FALSE` prints Case 1 step by step, `RaiOverlap` with `DISCHARGE = FALSE` prints Case 2, `RaiHandoff` with `RULE = FALSE` prints both Case 3 runs, and `RaiPrefix` with `FRESHCHILD = FALSE` or `GUARD = "target"` prints Case 4, `RaiRecovery` with `EXEMPT = TRUE` prints the short form of Case 5, and `RaiResolve` with `RULES = FALSE` prints Case 6. In the prototype, Cases 1, 2, Mixed NC, 3, and 4 exist as unit tests of the construction and of the voting rules, Case 5 as unit tests of the attachment rule (a child of a recovery-only tip is first-voted only once the tip is notarized in the child's epoch, and at once under a notarization lock), and Case 6 as unit tests of the termination rules (a witnessed rival of a kept block gets an instance but no vote; a contested position waits for the checkpoint; Rule 2 keeps the side with the most reporter votes, ties by hash); the benchmark harness's Byzantine validator votes at random and does not play these scripts, and a live harness with a scripted Byzantine validator that withholds named votes was not built. We consider this the main gap between the models and the measurements.

APPENDIX G
PROTOTYPE NOTES

A. Departures from the protocol

First, the manifest commits to signed votes and marks the closing-epoch witness of every overlap certificate its evidence proves: the closing-epoch exclusion witness, the current-epoch NC, a matching-origin witness for every record of the last closed state S_{e-2} the block bypasses, and a parent finalized in S_{e-1} or itself certified. The marks are a function of the evidence, the selection, and the two closed states, so a validator recomputes them and refuses a manifest that marks any other set. Blocks and ancestry are placed by the report items, and the witnesses that discharge records of the predecessor checkpoint are re-derived by every validator from the manifest's votes rather than bound to the claims they serve. Second, a first vote names as its base the state hash of the checkpoint its signer had installed, d_{e-1} for a settled vote and d_{e-2} for an early one, rather than a digest of the per-block admission evidence; a settled vote counts toward a fast certificate only at a validator that installed the checkpoint it names, and a validator that has not yet installed it holds the vote and counts it at installation. Third, retained vote records keep a fixed cap of four epochs, since the reports of the next three epochs may still cite an epoch's late finality, and beyond it only the support that may still discharge a surviving recovery record of its origin at its position. The handoff evidence, an epoch's frozen reports and their reconstructions, is released once members of the successor committee carrying $N-f$ of its weight acknowledge the checkpoint; a member acknowledges once the decided state is written and synced, empty checkpoints included, and unreleased epochs are kept up to sixteen, a forced drop being logged (none occurred in any run). Fourth, durability rests on the host's `fsync`: first, final, late, and close-election votes, frozen reports, overlap evidence, the epoch origin, and decided checkpoints are synced before any signature that depends on them leaves the process, but on the macOS host `fsync` hands the data to the drive without forcing its cache. A stricter mode issues `F_FULLFSYNC` after every signing write (about 4 ms each). Power loss was not simulated; the restart runs kill processes and test crash consistency only. Fifth, a position whose committed first votes are split so that no block can be notarized any more is not carried into the new epoch at the boundary; if the checkpoint then retains neither side, the position is voted afresh, and if it retains one side, that side is continued. Sixth, the *pause* transition used as a baseline in the evaluation is a configuration switch of the same code: under it no instance of epoch $e+1$ is started before S_e is decided locally, so the overlap rule and late notarization never apply; everything else, including reports, the construction, and installation, is unchanged. Seventh, a position counts as contested when any instance at its root holds two candidate blocks or the latest decided checkpoint retains an unresolved block there; a block forked at a different parent is not seen as contested until a checkpoint retains it. Eighth, the emulated link delay is applied per channel at the sender: every outbound message leaves its channel a fixed time after it was queued, in order, which approximates a one-way link latency without a network emulator. Ninth, for the comparison at Nano's quorum the equal-weight committee may have more members than $3f + 2p + 1$, every threshold following n ($n - f - p$ certify, $n - p$ finalize fast, $n - f$ reports), and an evaluation switch turns fast finalization certificates off; with $n = 6$, $f = 1$, $p = 0$ and no fast path a certificate needs five of six, as Nano's 67% quorum does. Tenth, the Byzantine validator of the evaluation is the same node built with an evaluation-only feature: it signs a second vote for the sibling of every fork block it votes for, sends a report with a corrupted residual

root to half of the validators, and, leading the first round of every close, proposes two values derived from two orders of the usable reports, voting for the second at once.

B. Defects found by the benchmarks

Defects (1) to (18) were found by the matrix of October 10, 2026 (nine variants at 2,000 blocks/s, three runs each, on a build before the re-notarization and termination rules) and by the harness runs of Appendix G-F; (19) to (22) by the review and an earlier throughput-latency sweep; (23) to (28) by the matrices of October 19 to 21, 2026, behind the main text, and (29) by its committee change with a Byzantine member. Several defects were found and repaired while measuring; none is a failure of the two services. (1) A validator records, for every block it votes for, the parent that block names, so that its residual records place the block; the record was dropped when the slot state of the older of two consecutive epochs that both voted the block was discarded. The newer report then committed a residual record with a zero parent, `BuildState` failed at every leader that selected it, and two variants never closed an epoch. The record now outlives every slot state that votes the block, and a report naming a block without its parent is malformed and never selected. (2) A block whose first votes split across an epoch boundary, three cast in the epoch left and two in the next, was retained as a recovery lock and never finalized; every unfinalized instance of the epoch left is now re-voted in the new epoch, and the generator implements the owner policy for the positions that remain. (3) Manifest chunk requests were flooded to every representative and the inbound queues overflowed; requests now go to the proposal's sender. (4) Re-voting every unfinalized instance filled the election container to its former cap of 5,000, after which no new block was activated on any validator; carried instances whose lock cannot continue are discarded at installation and the cap was raised. (5) Committee derivation deduplicated the epoch's finalized blocks quadratically, durable vote records shared the ledger's LMDB writer, and the predecessor gate was opened only after the committee bookkeeping, which together lengthened every wait at the gate. (6) Late notarization first shared a vote generator with the consensus instantiation's notarization votes and opened an instance of the closing epoch at its receivers for each late vote. (7) After the repairs of Appendix E, the manifest's entries grew to 72 bytes with the late and settled voter bit sets, a full manifest reply overflowed its 16-bit length field, and validators whose own manifest differed from the leader's silently refused every proposal; the chunk size was reduced. (8) A released epoch was recreated by a peer's repeated report and re-verified on every tick under the election container's lock, which stalled vote application for about a second after each installation; the exchange now remembers released epochs. (9) Copies of a proposal that leaders repeat every 200 ms were re-checked in full after the epoch was decided while they held the report-exchange lock, so inbound publishes were dropped and the p95 of one variant swung between 0.4 and 2.9 s between runs; a queued copy now returns once the value is held or the epoch decided.

The restart, rotation, and handoff runs of Appendix G-F found further defects. (10) The epoch origin T_0 was not persisted, so a restarted validator stayed in epoch 0; the origin, the genesis committee, the decided checkpoints, and the close votes are now durable records. (11) Once its peers had released an epoch's reports at $N - f$ acknowledgements, a validator one close behind could no longer reconstruct them and derive the checkpoint: it now fetches the value and state by the certificate's value hash from any holder; it also asks for the first rounds' statements of a close open for 2 s, ends its epoch once members holding more than f of the weight are seen voting in later epochs (otherwise a lagging replica leaves one epoch per boundary and never closes the gap), and, while its epoch has no election, asks once per epoch for the epoch's close. (12) Synced signing first added up to 440 ms to vote release, since the voter waited for the environment's single writer behind synced evidence batches and boundary deletions; decided states and evidence moved to a second environment and deletions to bounded transactions. (13) Rotation exposed four membership defects: a leader selected a non-member's report, which the manifest's member bit sets cannot name; a member whose weight had moved away stopped voting with its key and was tiered by weight; its votes were dropped by a weight gate; and a validator missing a frontier block's delegation derived another committee. Members are now known by membership, and derivation waits for the frontier blocks. (14) A position split three against three in epoch e, retained under recovery records of origin e, was finalized in $e+1$ by a changed committee: four of its members first-voted one side early, and after installing S_e the validators final-voted without rechecking eligibility, although no epoch-e witness discharged the rival's record. Every correct report then carried an `F` entry the report check refuses and the close stalled. The rule of Appendix B-B already forbids this final vote; the prototype now withholds it. (15) A validator that fell behind solicited more evidence, the replies overflowed its 64-message per-channel inbound queue, and among the drops were the few close-round votes it needed; with a Byzantine member every correct report is needed, so one straggler stalled the close. Close-round votes now bypass that queue, and members are solicited by membership. (16) The acknowledgement of an installed checkpoint was sent on the in-memory installation, never for a checkpoint that finalized nothing new, and the report exchange dropped epochs after four whether released or not.

Two prototype behaviours found by that matrix were not fixed at the time. (17) At a boundary the prototype re-votes in the new epoch every position at which a block can still be notarized, each validator voting the block it voted before. It decides this by counting the weight of members not seen voting as possibly still to come, so with a validator offline a position split three against two among the five running validators looks open: it is carried, the split repeats in the new epoch, and the minority's one first vote there is spent before the checkpoint that dropped their side is known. With all six running the same split is recognised and not carried,

and after the checkpoint every validator votes the kept block. (18) An epoch's timer starts with the first new election in it; positions carried into the epoch do not start it, so once the load stops no further checkpoint is decided. Together these leave the forked positions of some runs kept, identically everywhere, rather than finalized in that matrix. Neither affects safety. Behaviour (17) is what the second termination rule forbids, and the prototype now follows it; (18) no longer occurs, since carried and continued instances start the epoch's timer, and in the sweep every epoch after the load ended was closed.

A review after the matrix found one more defect, in the protocol and in the prototype alike. (19) A maximum-depth tip held by recovery records alone counted as a final parent, so a child of it was first-voted without a new notarization of the tip, and Case 5 finalized both sides of a fork. The prototype now requires such a tip to hold a notarization in the child's epoch, as for any other unresolved parent. The matrix runs predate the change; the load generator never builds on a forked position, so they never took the changed path. (20) The same review showed that a forked position could stay open forever (Appendix D-B); the prototype now implements the two termination rules and the new Rule 2. The matrix runs predate these rules as well. (21) With the termination rules, a first 10%-fork run left 1,505 positions open: an instance first-voted only the block its validator saw first, which rule (i) refuses where the checkpoint keeps the rival, and the kept block got an instance in the open epoch only at validators whose election held it as a saved candidate, not at those whose ledger took it in later or never had it. An instance now first-votes a valid candidate when its first-seen block is not valid, and every validator continues the kept block once its ledger holds it; the same run then settled with one position open. (22) Not fixed: with a committee change in every epoch and eight node processes on one host, a rotated-in validator fell far behind its peers at 1,000 blocks/s and above, in RAI and pause alike, so those runs did not settle in time. The single committee change of the main text, with six processes, settles at 2,000 blocks/s.

The matrices of the main text found six more. (23) The Byzantine validator signs one report for half of the validators and another for the rest. The report exchange kept the first version per reporter, so a proposal selecting the other version could never be checked at the validators holding the first: the check waited until the round timed out, and the validators that had voted for it proposed it again in their own rounds. At Nano's quorum every correct vote is needed, so closes took three or four rounds and held the epoch after next for 8 to 28 s. A proposal now carries the signature of every report it selects, as the protocol's decided value does with its report headers; a validator authenticates a version it lacks and reconstructs it next to the one it holds, keeping at most four versions per reporter and giving up a version no proposal could use yet for a newer one. The 95th percentile of non-fork blocks with a Byzantine validator fell from 5.7–18 s to 0.7–1.4 s without forks. (24) A late instance's discard rolls a fork block back from the ledger without changing the election's candidate entry, which still records it as saved; when the next checkpoint kept that block and every validator had rolled it back, no validator handed its body to the ledger and no ledger could serve it, and its position stayed open. Retained bodies are now taken from saved candidates too. (25) A validator that missed one final vote of a close round the others decided asked for the round's statements again, but its requests went through the same per-channel queues as its requests for the thousands of account positions it could not finish while lagging, and were dropped at every validator it asked; it never decided the epoch and its committee could not reach a quorum in the next one. Close-round requests now take the inbound queue's priority lane and a lane of the request aggregator. (26) Proposals are checked one at a time in arrival order, and a close round extended its timeout for a check only once the check found something to fetch, so the current leader's proposal could wait behind a Byzantine leader's two values while its round timed out. A round now waits from the moment its leader's proposal is queued, and the newest round's proposals are checked first; at Nano's quorum with 10% forks and a Byzantine validator, five pairs of runs of the same seeds gave a median of 1,443 against 1,265 non-fork blocks/s and a 95th percentile of 4.7 against 9.1 s. (27) The load generator published the blocks that move weight for a committee change to the first validator only, and a validator that missed its flood never received them under Nano, which follows no checkpoint; they now go to every validator, as the load does.

Two changes were tried and reverted, and the behaviour they addressed remains. (28) When closes take longer than an epoch, a forked position carried by the checkpoint can be voted in the next epoch only once that checkpoint is installed, and the epoch is left 2 s after the installation: in one of nine runs with a committee change and 10% forks, closes fell further behind at each close, every report re-justified the carried records, and 704 positions stayed open until the harness stopped waiting; the same seed resolved every fork when run again. With a Byzantine member that keeps its seat through the change, one of ten 10%-fork runs left 321 positions open in the same way: each close after the load took 8 to 18 s, longer than the epoch, since the Byzantine member leads its first round. Holding an epoch open until its carried positions terminate stalled new blocks for the length of the hold with a Byzantine validator. Separately, a validator that lags is flooded with live votes and drops the votes it asked for as evidence of other reports' entries, which delays its readiness for a close by up to 20 s; giving those replies a lane of their own aligned the validators' readiness but crowded out the live votes and lowered the confirmation rate.

The committee change with a Byzantine member that keeps its seat found one more. (29) The repair of (24) took a retained block's body from an election that still held it. A late instance's discard also rolls back what was built on the discarded block, and those instances are erased: when a later checkpoint kept one of these blocks and every validator had rolled it back, no election and no ledger held it, and 27 positions stayed open in one 5%-fork run. The validator now keeps the bodies of the

blocks a late instance's discard rolls back, forces a retained one in from them, and answers other validators' requests for it. Keeping the body of every rolled-back block instead was worse: a fork loser forced back in fights the live instance resolving its position, and on the 10%-fork cell that variant processed three to four times as many blocks, confirmed less than the build before the repair on four of five seeds, and left one run unsettled.

C. Throughput and latency over time with a faulty validator

These figures are from the matrix of October 10, 2026, on the build before the re-notarization and termination rules, at an offered 2,000 blocks/s; the main text no longer reports that matrix. They show the variants with one validator offline or Byzantine (the latter voting at random, without leading closes) second by second: left, non-fork blocks confirmed per second (clipped at 4,000); right, median (solid) and 95th-percentile (dotted) latency of the blocks confirmed in each second; the bars at the top mark when RAI and pause are closing an epoch. Each panel shows the run with the median 95th percentile of three; a pause run without client numbers is never chosen.

D. What late notarization buys

On the build of October 7, 2026, before the repairs of Appendix E, the nine-variant matrix was run twice on the same day: with the core prefix-wide overlap rule alone and with late notarization added. Without late notarization a block published during a close has no closing-epoch exclusion witness, so every such block waited for the checkpoint: 1,900 to 7,100 instances per boundary, client confirmations stopped for one to three seconds at each close, p95 was 1.3 to 2.4 s without forks and 2.1 to 8.2 s with forks, and 10%-fork goodput was 15 to 25% lower. With late notarization p95 fell four- to eightfold in every variant (no forks: 305, 176, and 186 ms for the all-online, one-offline, and one-Byzantine variants) while medians stayed within 10 ms. Late notarization keeps the core rule and its proof, costs one more signature per block and committee member during each close, and needs no members common to the two committees. The pause baseline of the main text's evaluation is the other end of the same scale: no early votes at all.

E. Committee change

A first committee model keeps the same six members and recomputes their weights from the finalized state at each checkpoint within $\pm 10\%$ of the equal share; every validator derived the same committee digests, and the no-fork variants settled with latencies close to the equal-weight runs. The equal-weight model re-selects the heaviest N representatives from the finalized state, so committee identities change when weight moves. The harness runs extra representatives outside the committee and, at the start of a chosen epoch k, moves a member's whole balance to a holder delegating to a standby representative; the committee derived from epoch k, used from epoch $k + 2$, has the standby in the member's place. At 800 blocks/s, 20,000 blocks, and 5% forks, with eight validators of which six form the committee: replacing two members settled in 3 of 3 runs (667 to 748 non-fork blocks/s, p50 103 to 106 ms); replacing two members and restoring them two epochs later with a Byzantine member in every committee settled in 2 of 2 runs (578 and 581 blocks/s, p95 385 and 572 ms); and with a four-member committee ($f = 1$, $p = 0$) whose three correct members were all replaced while the Byzantine member stayed, 1 of 1 (728 blocks/s, p50/p95 208/337 ms). Every validator derived the same members for every epoch, checked by listing them, and no run produced a conflicting signature. These counts are after the repairs (13) to (15) above; before them 2 of 5 two-member runs and every Byzantine run stalled at the first close after the change. Ten validators were more than the host could carry, hence the four-member committee for the full replacement. These runs are from the build of October 9, 2026 (commit `4582bf003` and later), one day before the October 10 matrix. The main text's committee change (its Figure 7) is a later sweep: six validators, four seats ($f = 1$, $p = 0$, no fast path), two members replaced in epoch 0, at 2,000 blocks/s with RAI, pause, and Nano, three runs each (`run_sweep.py -set rotation`); the runs with a Byzantine member use `-set rotation-byz`, which moves the weight of two correct members so that the Byzantine member keeps its seat.

F. Restart, durable signing, and handoff

Every node logs each signature it releases to a file before sending it. The harness kills validators with `SIGKILL`, either at a fixed time into the load, while elections are running, or when a given close starts, restarts them on their data after a chosen downtime, and afterwards checks every logged signature of every node: no identity may have signed two blocks in one first-vote or final-vote instance or two values in one close round. Fork5 with two restarts, one at a close and one mid-load, settled twice with every checkpoint decided identically on all six validators and no conflicting signature; the restarted validators rejoined through checkpoint catch-up and follow-ahead.

Synced signing costs latency, not goodput. Fork5, six validators, 45,000 blocks at 2,000 blocks/s, two runs per setting: without sync 1,765 and 1,820 non-fork blocks/s at p50 173 and 139 ms; with `fsync` 1,802 and 1,688 at 282 and 310 ms; with `F_FULLFSYNC` 1,763 and 1,777 at 409 and 304 ms. The evaluation of the main text runs with `fsync`.

Handoff with a crashing departing member: a member replaced at the rotation of epoch 1 is killed when the close of epoch 2, the last it decides, starts, and stays down 30 s; eight validators, 800 blocks/s, 20,000 blocks, 5% forks. Without and with a Byzantine member every run settled with every checkpoint decided identically (epochs 0 to 4 and 0 to 5), the restarted member caught up the epochs it missed and installed them, every epoch's handoff evidence was released only after $N - f$ durable acknowledgements, no epoch was dropped before release, and no signature conflicted (340 and 335 non-fork blocks/s; the client counts the outage).

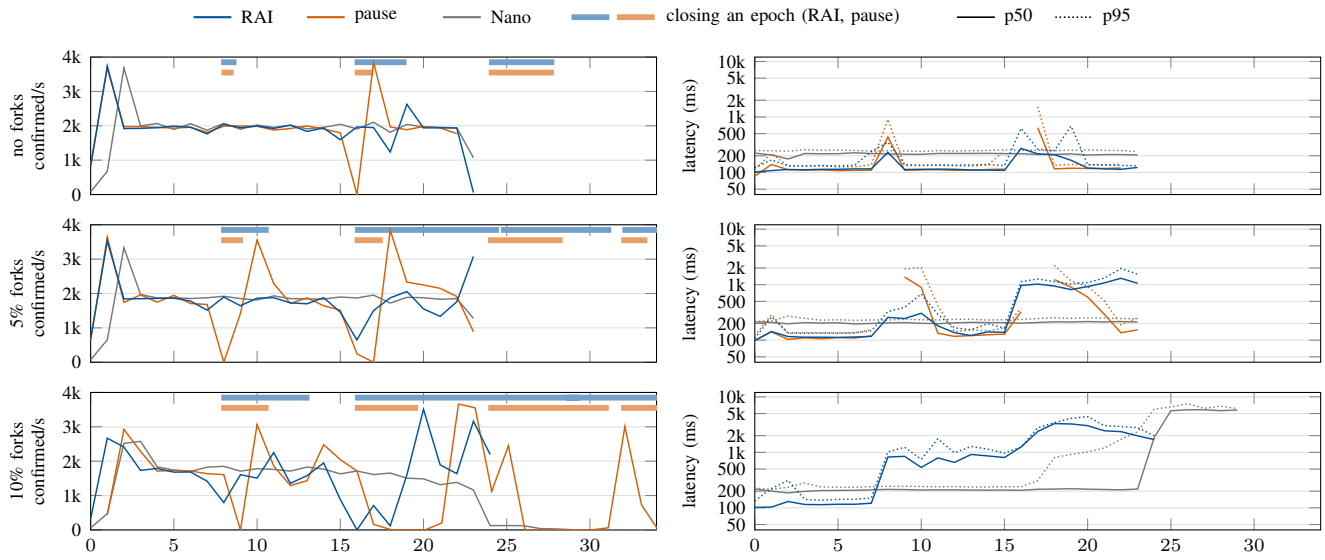


Fig. 1. Throughput and latency over time with one validator offline, on loopback (October 10 build). Pause has no client numbers at 10% forks: in all three runs its client never saw every non-fork block confirmed, so the plotted run, the one that got furthest, shows the client's per-second count of all confirmations (forks included) and no latency.

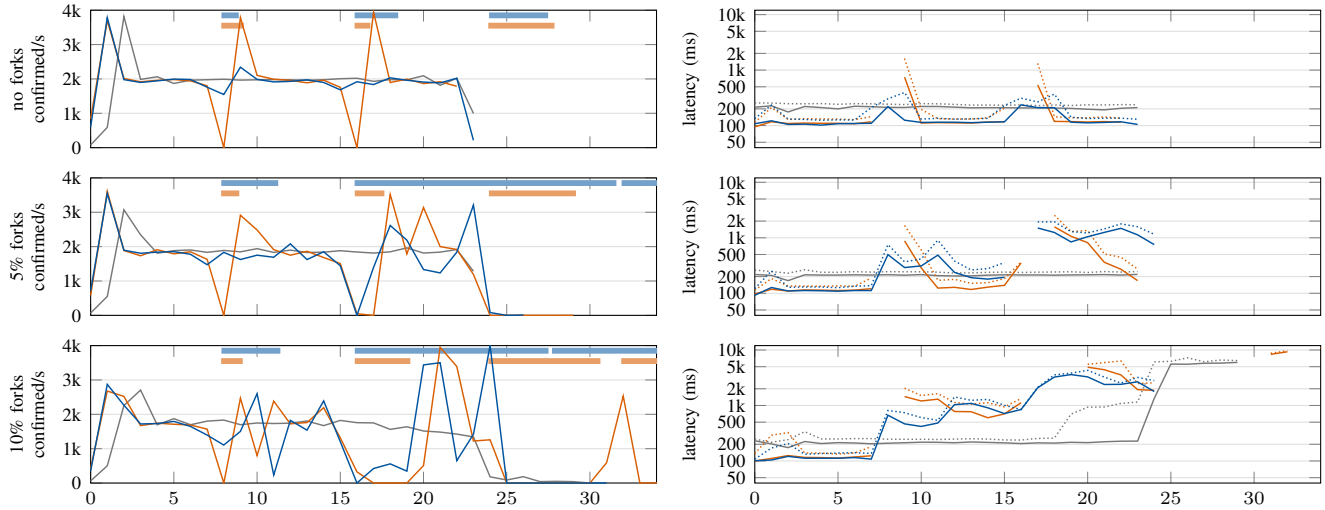


Fig. 2. Throughput and latency over time with one Byzantine validator voting at random, on loopback (October 10 build).

Committing the overlap marks and naming the vote base were each compared against the build before them, two fork5 runs and one fork5 run with a Byzantine member per arm, alternating. Neither changed goodput, latency, or close time beyond the spread between runs of the same build (1,510 to 1,792 non-fork blocks/s, p50 167 to 332 ms across the twelve runs); checking the marks adds 30 to 60 ms to a follower's evidence check, and the base moves about 10% of fast finalizations to final votes, since a settled vote whose base a receiver has not installed no longer counts there. About 20,000 of the 25,000 blocks of a fork5 run finalize under the overlap exception, and the closes mark 5,000 to 14,000 certificates per epoch.

G. Reproduction

The prototype, the harness, the TLA$^+$ models, and every run record (the per-run `result.json` with the client's per-second series, its latency histograms by publication second, and the nodes' epoch events, the node configurations, and the logs) are in the artifact. The matrices of the main text are produced by

```
python3 tools/rai/run_sweep.py <dir>
-set final
-rate 2000 -blocks 48000 -reps 3
-arm rai=<bin>:-committee-p=0:-committee-seats=6
-arm pause=<bin>:-committee-p=0:-committee-seats=6:
-epoch-transition=pause
-arm develop=<develop bin>:-no-epochs
```

with `RAI_FAST_PATH=0` in the environment of the

RAI and pause nodes; without the committee flags and that variable for $f = p = 1$; and with `-set rotation` and `-set rotation-byz` for the committee change. Every arm's binary directory holds `rsnano`, a `rsnano-byzantine` built with the `rai_byzantine` feature, and the generator, all in release mode with the `rai_protocol` feature; Nano is the develop branch with the fork-signature check (`develop-fork-memo`). The sets seed the generator identically for every system (`-seed`), run the Byzantine node (`-byzantine-node equivocate,conflicting,equivocate`) as the first leader of every close (`-byzantine-leads-close`), and keep the nodes running after settling until no forked position is open (`-fork-grace`). One point alone is `tools/rai/run_gate_b.py <dir> -rate 2000 -fork-percentage 10`.

The flags `-restart PR:close:E`, `-restart PR:at:S`, `-restart-down-ms`, `-audit-votes`, `-signing-sync`, `-prs`, `-standby`, and `-rotation E:FROM>TO` drive the runs of Appendices G-E and G-F. The figures of the main text are regenerated with `tools/matrix_figures.py -p0 <dir> -p1 <dir> -rotation <dir> -rotation-byz <dir> sections/eval-data/matrix`; the October 10 matrix behind Figures 1 and 2 with `tools/rai/run_eval_matrix.py` and `tools/eval_figures.py`. The models are checked with `tla/run.sh <module> <config>` and the scripts in `tla/`.

TABLE II

TLC RUNS. TIMES ARE WALL-CLOCK ON TWO CORES; *EIGHT CORES; †SIX WORKERS. SIM IS RANDOM SIMULATION (A VIOLATION FOUND BY SIMULATION IS A REAL TRACE; NO VIOLATION WOULD PROVE NOTHING).

Configuration	Result	Time
`Close_FALSE`	`ClosureThm1` violated after 19 steps	35 s
`Close_FALSE_conflict`	`NoConflict` violated after 23 steps	2 min
`Close_TRUE`	no error; 651,632 distinct states	3 min
`Overlap_fixed`	no error; 5,217,510 distinct states	14 min
`Overlap_noDischarge`	`SameVerdict` violated after 15 steps	1 min
`Overlap_noLateEv`	`WitnessAdmissible` violated after 14 steps	1 min
`Overlap_noSplit`	`UsableReports` violated after 9 steps	10 s
`Overlap_noSplit_safety`	other four hold; 5,485,444 states	14 min
`Overlap_probe1`, `_probe2`	probes violated, as intended	$<$ 1 min
`Handoff_same_paper`	`ClosureThm1b` violated after 22 steps	11 min
`Handoff_disjoint_paper`	`ClosureThm1b` violated after 23 steps	15 min
`Handoff_same_rule`	no error; 8,347,138 states	15 min
`Handoff_disjoint_rule`	no error; 3,707,876 states	17 min
`Handoff_overlap3_rule`	no error; 121,053,272 states	2 h 18 min*
`Handoff_Probe*`	probes violated, as intended	$<$ 5 min
`Prefix4_same_{fixed,target,nofresh}`	no error; 278,534 states each	10 s*
`Prefix4_disjoint_fixed`, `_target`	no error; 1,622,276 states	1.7 min*
`Prefix4_disjoint_nofresh`	`NoConflict` violated after 12 steps	5 s*
`Prefix4_shared1_fixed`, `_target`	no error; 3,960,746 states	3.5 min*
`Prefix4_shared1_nofresh`	`NoConflict` violated after 12 steps	8 s*
`Prefix4_shared2_fixed`	no error; 2,434,078 states	1.7 min*
`Prefix4_shared2_target`	`NoConflict` violated after 13 steps	19 s*
`Prefix4_shared2_nofresh`	`NoConflict` violated after 14 steps	12 s*
`Prefix4_*_probes`, `Prefix4_shared2_Probe*`	probes violated, as intended	$<$ 2 s*
`Prefix6_disjoint_nofresh_sim`, `Prefix6_shared3_target_sim`, `_nofresh_sim`	`NoConflict` violated (simulation)	$<$ 1 s*
`Prefix6_same_probes`, `Prefix6_shared3_probes`	probes violated, as intended	14 s, 3.7 min*
`Prefix6_same_fixed`, `Prefix6_shared3_target` (BFS)	unfinished at submission: 13.4 M and 35.8 M distinct states explored, no violation so far	–
`Recovery4_same_exempt`	`NoConflict` violated after 16 steps	6 s†
`Recovery4_{disjoint,shared1,shared2}_exempt`	`NoConflict` violated after 16 steps	$<$ 1 min†
`Recovery4_same_fixed`	no error; 734,980 states	17 s†
`Recovery4_disjoint_fixed`	no error; 755,978 states	42 s†
`Recovery4_shared1_fixed`	no error; 3,337,378 states	2.5 min†
`Recovery4_shared2_fixed`	no error; 3,922,566 states	2.2 min†
`Recovery4_same_Probe*`	probes violated, as intended	$<$ 5 s†
`Recovery6_same_exempt_sim`	`NoConflict` violated (simulation)	$<$ 1 s†
`Recovery6_same_ProbeRecovery`	probe violated (simulation)	$<$ 1 s†
`Recovery6_same_fixed` (BFS)	unfinished (stopped after 75 min): 87.5 M distinct states to depth 25, no violation	–
`Close_TRUE_maxkeep`	no error; 278,404 states	1.1 min†
`Overlap_fixed_maxkeep`	no error; 4,959,763 states	6.3 min†
`Handoff_same_rule_maxkeep`, `_disjoint_rule_maxkeep`	no error; 5,862,017 and 2,686,030 states	9.5, 10.6 min†
`Recovery4_{same,disjoint,shared1,shared2}_fixed_maxkeep`	no error; 0.6–3.2 M states	$<$ 3 min†
`Resolve4_norules_sim`, `_norules_23_sim`	`TwoGood` violated (simulation)	$<$ 2 s†
`Resolve4_rules_sim`	no violation in 493,465 random runs (88 M states)	15 min†
`Resolve4_rules_23`	no error, no deadlock; 4,635,660 states, depth 33	8.8 min†
`Resolve4_rules_23_ProbeOneGood`, `_ProbeGood23`	probes violated, as intended (10 and 23 steps)	$<$ 2 min†